UNIVERSITY OF THE PEOPLE

CS 1101 Unit 3

Programming Assignment

Assignment Response

1. **Circumference Calculation**

The following is the countdown function copied from Section 5.8 of your textbook.

```
def countdown(n):
    if n <= 0:
        print('Blastoff!')
    else:
        print(n)
        countdown(n-1)
```

Write a new recursive function countup that expects a negative argument and counts “up” from that number. Output from running the function should look something like this:

```
>>> countup(-3)
-3
-2
-1
```

Blastoff!

Write a Python program that gets a number using keyboard input. (Remember to use input for Python 3 but raw_input for Python 2.)

If the number is positive, the program should call countdown. If the number is negative, the program should call countup. Choose for yourself which function to call (countdown or countup) for input of zero.

Provide the following.

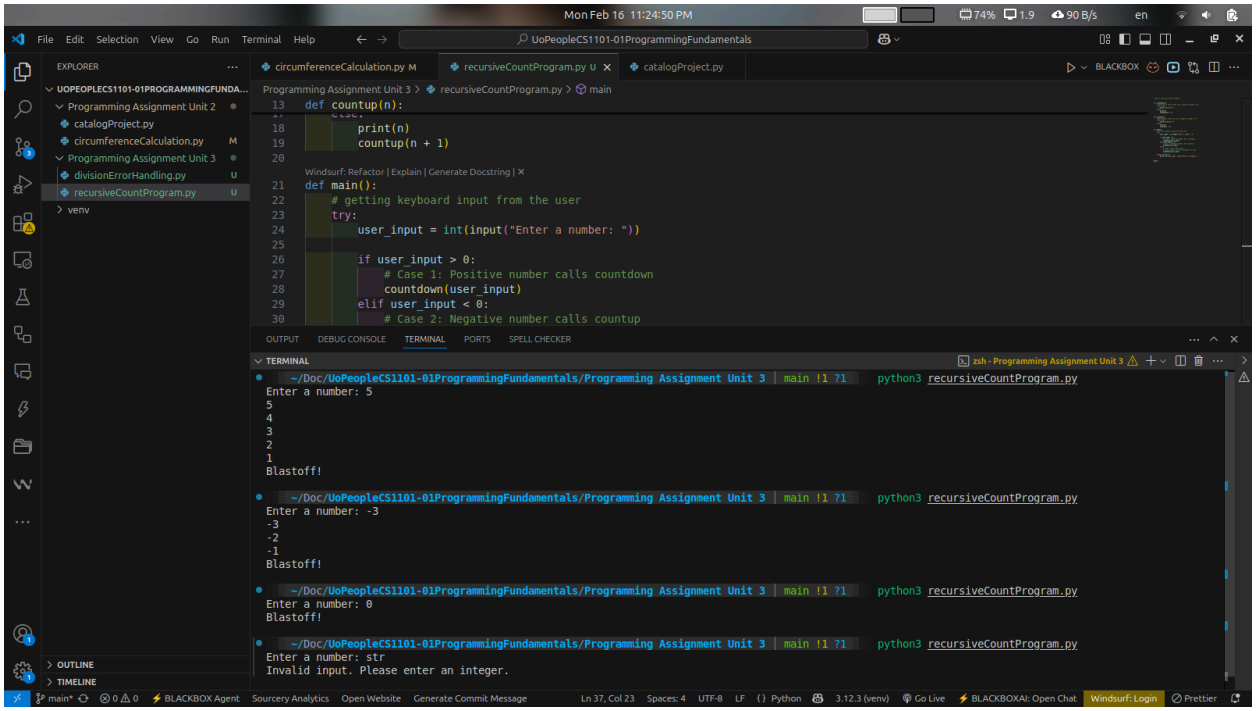
- The code of your program.
- Respective output for the following inputs: a positive number, a negative number, and zero.
- An explanation of your choice for what to call for input of zero.

In programming, a function is a named sequence of statements that performs a computation (Downey, 2015). A function that calls itself is recursive, and the process of executing it is called recursion (Downey, 2015).

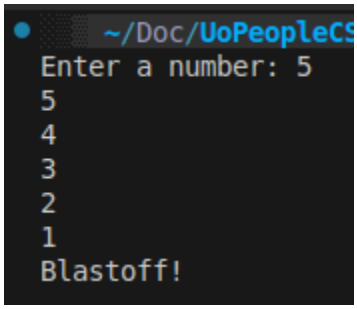
```
        # Part 1: Recursive Count Program

def countdown(n):

    # Recursively counts down from a positive number to 0.

    if n <= 0:
```

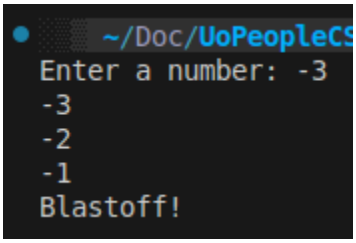
```
        print("Blastoff!")
    else:
        print(n)
        countdown(n - 1)


def countup(n):
    # Recursively counts up from a negative number to 0.
    if n >= 0:
        print("Blastoff!")
    else:
        print(n)
        countup(n + 1)


def main():
    # getting keyboard input from the user
```

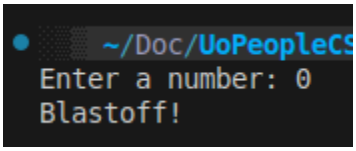
```
    try:
        user_input = int(input("Enter a number: "))
        if user_input > 0:
            # Case 1: Positive number calls countdown
            countdown(user_input)
        elif user_input < 0:
            # Case 2: Negative number calls countup
            countup(user_input)
        else:
            # Case 3: Zero input logic
            # I have chosen to call the countdown for zero
            countdown(user_input)
    except ValueError:
        print("Invalid input. Please enter an integer.")
main()
```



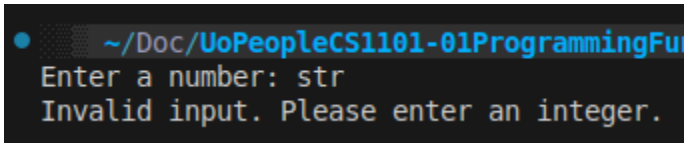
Here is the output generated by the program for the three required scenarios:







To prevent this crash, we use a try…except block. This tells Python: “Try to convert this input, but if it fails because the user typed a string, don't crash - just run this 'emergency' code instead.”



Explanation of Choice for Zero

For an input of zero, I chose to call the ***countdown*** function.

Reasoning:

As noted in the textbook, “Every time a function gets called, Python creates a frame to contain the function’s local variables” (Downey, 2015). For these recursive functions, multiple frames exist on the stack simultaneously until the base case is reached. If a recursion never reaches a base case, it results in infinite recursion, which is “generally not a good idea” as it leads to a maximum recursion depth error (Downey, 2015).

For an input of zero, I chose to call the ***countdown*** function.

This decision is based on both logical consistency and the structure of the recursive base case. In the countdown function, the base condition is defined as n <= 0. This means that zero is already included in its stopping condition. When zero is passed, the function immediately reaches the base case and prints "Blastoff!" without making further recursive calls. In contrast, the countup function is specifically designed to handle negative numbers and moves toward zero by increasing the value (n + 1). Since zero is neither negative nor moving upward toward zero, it does not logically belong to the upward-counting process. From a conceptual perspective, zero represents the end of a countdown. In real-world countdown scenarios (such as rocket launches), reaching zero signals completion. Therefore, calling countdown(0) aligns naturally with both the recursive logic and real-life meaning of a countdown. Additionally, choosing countdown ensures clarity and avoids unnecessary recursion. Because zero immediately satisfies the base case

condition, the function terminates safely and efficiently, reinforcing the importance of properly defined base cases in recursion.

2. Division Error Handling

You are developing a program that performs a division operation on two numbers provided by the user. However, there is a situation where a runtime error can occur due to a division by zero. To help junior developers learn about error handling in expressions and conditions, you want to create a program deliberately containing this error and guide them in diagnosing and fixing it.

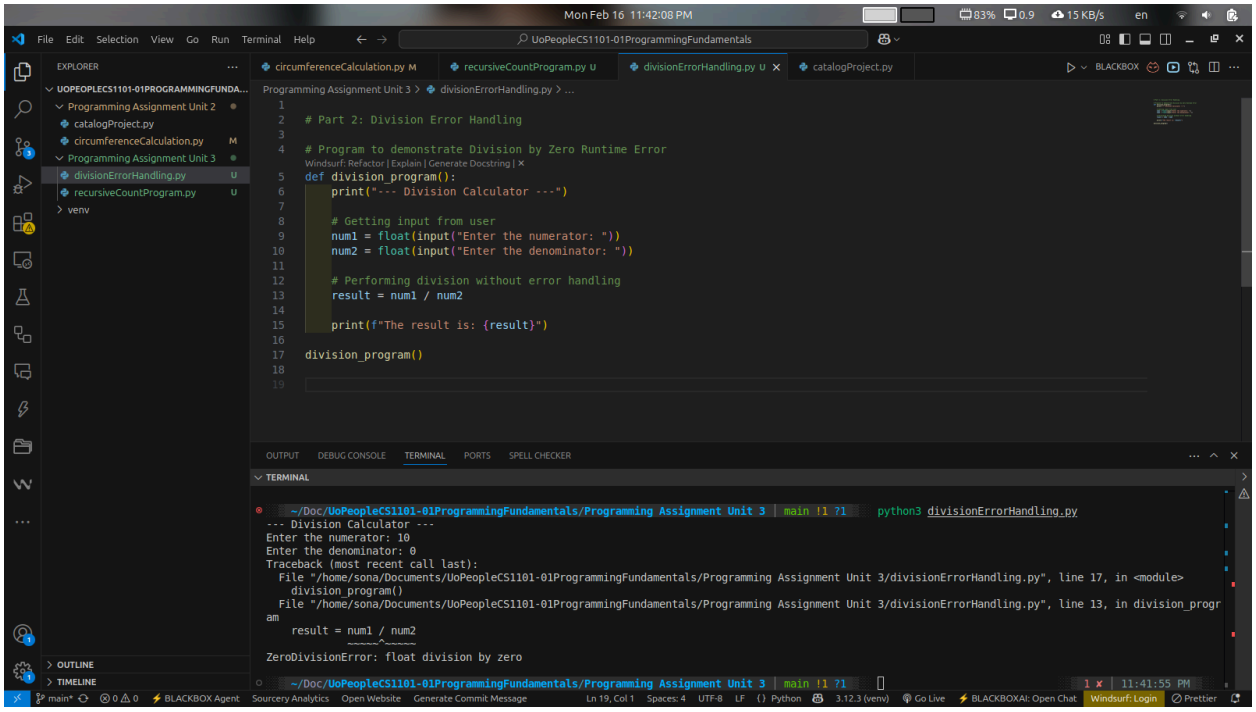
This program demonstrates a Runtime Error, which is an error that *"does not appear until after the program has started running"* (Downey, 2015).

```
# Program to demonstrate Division by Zero Runtime Error

def division_program():
print("--- Division Calculator ---")

# Getting input from user
num1 = float(input("Enter the numerator: "))
num2 = float(input("Enter the denominator: "))

# Performing division without error handling
result = num1 / num2
```

```
    print(f"The result is: {result}")


division_program()
```

When the denominator is zero, Python produces a traceback, which is a *"list of functions"* executing when the error occurred (Downey, 2015).

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 3 | main !1 ?1   python3 divisionErrorHandling.py
--- Division Calculator ---
Enter the numerator: 10
Enter the denominator: 0
Traceback (most recent call last):
  File "/home/sona/Documents/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 3/divisionErrorHandling.py", line 17, in <module>
    division_program()
  File "/home/sona/Documents/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 3/divisionErrorHandling.py", line 13, in division_
am
    result = num1 / num2
             ~~~~^~~~~
ZeroDivisionError: float division by zero
```



Understanding the Error Types

It is important to distinguish this from other errors. Syntax errors indicate something is wrong with the "structure of the program" and are found during translation (Downey, 2015). Semantic errors, on the other hand, happen when the program runs but "doesn't do the right thing" because the logic is flawed (Downey, 2015).

A ZeroDivisionError is a Runtime Error (also called an exception) because it happens while the program is running. As Downey (2015) points out, the message tells you "where Python noticed a problem, which is not necessarily where the error is". In this case, the error is caused by unchecked user input.

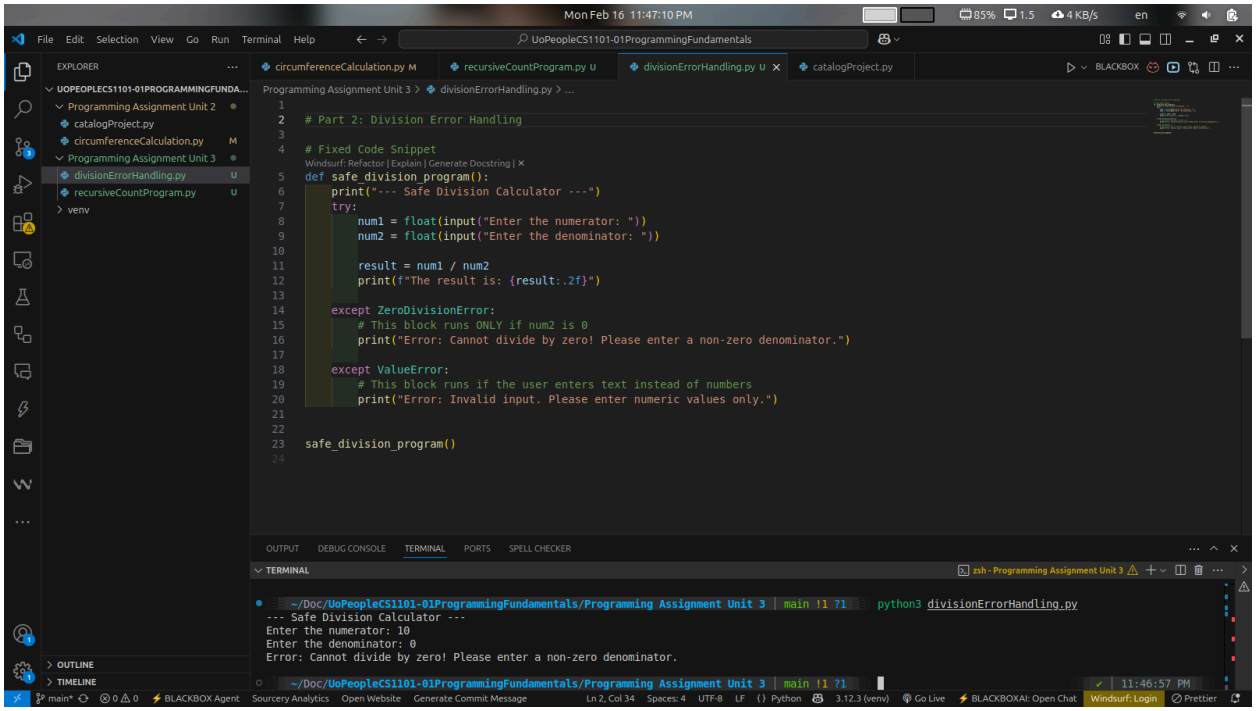
The Solution: Safe Error Handling

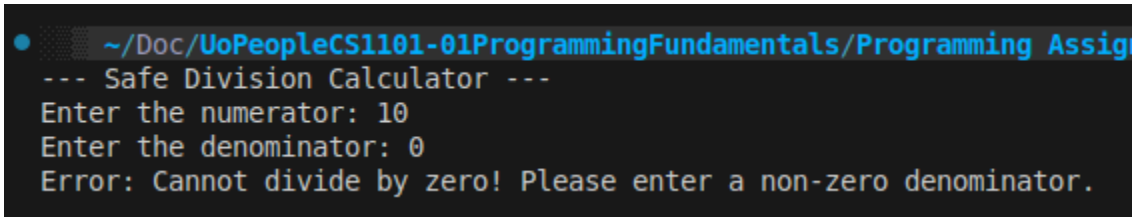
To prevent the program from crashing, we use a try...except block. This ensures that if an exception occurs, the program can provide a helpful message instead of halting.

Fixed Code Snippet:

```
    # Fixed Code Snippet

def safe_division_program():

    print("--- Safe Division Calculator ---")
```

```
    try:
        num1 = float(input("Enter the numerator: "))
        num2 = float(input("Enter the denominator: "))

        result = num1 / num2
        print(f"The result is: {result:.2f}")

    except ZeroDivisionError:
        # This block runs ONLY if num2 is 0
        print("Error: Cannot divide by zero! Please enter a   non-zero denominator.")

    except ValueError:
        # This block runs if the user enters text instead of numbers
        print("Error: Invalid input. Please enter numeric values only.")

safe_division_program()
```





References

Downey, A. (2015). Think Python: How to think like a computer scientist. Green Tree Press.

https://greenteapress.com/thinkpython2/thinkpython2.pdf